Experiment 7 — Normalization (UNF → 3NF → BCNF)

Aim

Given a denormalized Sales table, identify functional dependencies and normalize step by step from UNF to 3NF and BCNF. Verify lossless join and dependency preservation using SQL queries.

Source Code

Step 1 — Unnormalized (UNF) table

sql

```
-- ============================================================
-- Experiment 07 : Normalization UNF -> 3NF -> BCNF
-- MySQL 8.0
-- ============================================================
DROP DATABASE IF EXISTS NormalizeDB;
CREATE DATABASE NormalizeDB;
USE NormalizeDB;


-- ------------------------------------------------------------
-- UNF : repeating groups, mixed entities, redundant data
-- ------------------------------------------------------------
CREATE TABLE Sales_UNF (
    sale_id         INT,
    customer_id     INT,
    customer_name   VARCHAR(50),
    customer_city   VARCHAR(50),
    product_id      INT,
    product_name    VARCHAR(50),
    product_category VARCHAR(50),
    seller_id       INT,
    seller_name     VARCHAR(50),
    seller_city     VARCHAR(50),
    qty             INT,
```

```
    unit_price      DECIMAL(10,2),
    sale_date       DATE
);


INSERT INTO Sales_UNF VALUES
(1,101,'Ravi','Delhi',
1001,'Laptop','Electronics',201,'TechnoMart','Bengaluru',2,55000,'2024-01-05'),
(1,101,'Ravi','Delhi',   1002,'Mouse' ,'Accessories',201,'TechnoMart','Bengaluru',3,
500,'2024-01-05'),
(2,102,'Sita','Mumbai',
1001,'Laptop','Electronics',201,'TechnoMart','Bengaluru',1,55000,'2024-01-08'),
(3,103,'Arjun','Chennai',1003,'Shirt' ,'Clothing'   ,202,'StyleHub' ,'Pune'     ,5,
800,'2024-01-12'),
(4,101,'Ravi','Delhi',   1003,'Shirt' ,'Clothing'   ,202,'StyleHub' ,'Pune'     ,2,  800,'2024-
01-15'),
(5,104,'Meera','Kolkata',1004,'Novel' ,'Books'      ,203,'BookBazaar','Delhi'   ,4,
400,'2024-01-20');
```

Step 2 — Identify Functional Dependencies

text

```
FD1: (sale_id, product_id) -> qty, unit_price, sale_date
FD2: sale_id              -> customer_id
FD3: customer_id          -> customer_name, customer_city
FD4: product_id           -> product_name, product_category, seller_id
FD5: seller_id            -> seller_name, seller_city
```

Problem: FD2, FD3, FD4, FD5 are partial/transitive dependencies → not in 2NF, 3NF, BCNF.

Step 3 — 1NF

A relation is in 1NF when every attribute is atomic and there are no repeating groups. The Sales_UNF table already has atomic values (each row = one product in one sale). So the schema above **is in 1NF** — but it has partial and transitive dependencies.

Step 4 — 2NF (remove partial dependencies)

The candidate key is (sale_id, product_id). Attributes depending on only part of the key must be moved out.

sql

```
-- 2NF decomposition
CREATE TABLE Sale_2NF (
    sale_id     INT,
    customer_id INT,
    sale_date   DATE,
    PRIMARY KEY (sale_id)
);

CREATE TABLE Product_2NF (
    product_id       INT,
    product_name     VARCHAR(50),
    product_category VARCHAR(50),
    seller_id        INT,
    PRIMARY KEY (product_id)
);

CREATE TABLE SaleItem_2NF (
    sale_id    INT,
    product_id INT,
    qty        INT,
    unit_price DECIMAL(10,2),
    PRIMARY KEY (sale_id, product_id)
);
```

Step 5 — 3NF (remove transitive dependencies)

customer_name, customer_city depend on customer_id (not directly on sale_id). Same for seller_name, seller_city via seller_id.

sql

```
-- 3NF decomposition
CREATE TABLE Customer (
    customer_id   INT PRIMARY KEY,
```

```
    customer_name VARCHAR(50),
    customer_city VARCHAR(50)
);

CREATE TABLE Seller (
    seller_id   INT PRIMARY KEY,
    seller_name VARCHAR(50),
    seller_city VARCHAR(50)
);

CREATE TABLE Product (
    product_id       INT PRIMARY KEY,
    product_name     VARCHAR(50),
    product_category VARCHAR(50),
    seller_id        INT,
    FOREIGN KEY (seller_id) REFERENCES Seller(seller_id)
);

CREATE TABLE Sale (
    sale_id     INT PRIMARY KEY,
    customer_id INT,
    sale_date   DATE,
    FOREIGN KEY (customer_id) REFERENCES Customer(customer_id)
);

CREATE TABLE SaleItem (
    sale_id    INT,
    product_id INT,
    qty        INT,
    unit_price DECIMAL(10,2),
```

```
    PRIMARY KEY (sale_id, product_id),
    FOREIGN KEY (sale_id)    REFERENCES Sale(sale_id),
    FOREIGN KEY (product_id) REFERENCES Product(product_id)
);
```

Step 6 — BCNF check

BCNF: every determinant must be a candidate key. In the 3NF schema above:

Relation	FDs	Determinant a candidate key?
Customer	customer_id → customer_name, customer_city	✅ (PK)
Seller	seller_id → seller_name, seller_city	✅ (PK)
Product	product_id → product_name, product_category, seller_id	✅ (PK)
Sale	sale_id → customer_id, sale_date	✅ (PK)
SaleItem	(sale_id, product_id) → qty, unit_price	✅ (PK)

The 3NF schema is already in **BCNF** — no further decomposition needed.

Step 7 — Populate normalized tables

sql

```
INSERT INTO Customer VALUES
(101,'Ravi','Delhi'),(102,'Sita','Mumbai'),
(103,'Arjun','Chennai'),(104,'Meera','Kolkata');

INSERT INTO Seller VALUES
(201,'TechnoMart','Bengaluru'),(202,'StyleHub','Pune'),
(203,'BookBazaar','Delhi');
```

```
INSERT INTO Product VALUES
(1001,'Laptop','Electronics',201),
(1002,'Mouse','Accessories',201),
(1003,'Shirt','Clothing',202),
(1004,'Novel','Books',203);

INSERT INTO Sale VALUES
(1,101,'2024-01-05'),(2,102,'2024-01-08'),
(3,103,'2024-01-12'),(4,101,'2024-01-15'),
(5,104,'2024-01-20');

INSERT INTO SaleItem VALUES
(1,1001,2,55000),(1,1002,3,500),
(2,1001,1,55000),
(3,1003,5,800),(4,1003,2,800),
(5,1004,4,400);
```

Step 8 — Lossless join verification

sql

```
-- Row counts should match between UNF and reconstructed view
SELECT COUNT(*) AS unf_rows FROM Sales_UNF;

SELECT COUNT(*) AS reconstructed_rows
FROM Sale s
JOIN SaleItem si ON s.sale_id     = si.sale_id
JOIN Customer c  ON s.customer_id = c.customer_id
JOIN Product p   ON si.product_id = p.product_id
JOIN Seller sl   ON p.seller_id   = sl.seller_id;
```

sql

```
-- Compare the actual data (all columns)
SELECT s.sale_id, c.customer_id, c.customer_name, c.customer_city,
```

```
    p.product_id, p.product_name, p.product_category,
    sl.seller_id, sl.seller_name, sl.seller_city,
    si.qty, si.unit_price, s.sale_date
FROM Sale s
JOIN SaleItem si ON s.sale_id     = si.sale_id
JOIN Customer c  ON s.customer_id = c.customer_id
JOIN Product p   ON si.product_id = p.product_id
JOIN Seller sl   ON p.seller_id   = sl.seller_id
ORDER BY s.sale_id, p.product_id;
```

Step 9 — Dependency preservation check

Run each FD against its decomposed relation to confirm no tuple violates it.

sql

```
-- FD3 preserved in Customer
SELECT customer_id,
    COUNT(DISTINCT customer_name) AS name_variants,
    COUNT(DISTINCT customer_city) AS city_variants
FROM Customer
GROUP BY customer_id
HAVING name_variants > 1 OR city_variants > 1;
-- Expected: empty (each customer_id maps to exactly one name & city)

-- FD4 preserved in Product
SELECT product_id,
    COUNT(DISTINCT product_name)     AS name_variants,
    COUNT(DISTINCT product_category) AS cat_variants,
    COUNT(DISTINCT seller_id)        AS seller_variants
FROM Product
GROUP BY product_id
HAVING name_variants > 1 OR cat_variants > 1 OR seller_variants > 1;
-- Expected: empty
```

```
-- FD5 preserved in Seller
SELECT seller_id,
    COUNT(DISTINCT seller_name) AS name_variants,
    COUNT(DISTINCT seller_city) AS city_variants
FROM Seller
GROUP BY seller_id
HAVING name_variants > 1 OR city_variants > 1;
-- Expected: empty
```

Output

Query	Result
COUNT(*) FROM Sales_UNF	6
COUNT(*) reconstructed	6
Reconstructed row-by-row	Matches Sales_UNF exactly
FD3 / FD4 / FD5 preservation queries	0 rows each (no violations)

Conclusion: All functional dependencies are preserved across the decomposed tables, and the natural join of the 3NF/BCNF schema produces exactly the original UNF data — **lossless and dependency-preserving**.